

Naive Bayes Classifier

Juan G. Colonna
juancolonna@gmail.com

Naive Bayes (NB)

Os modelos baseados em Naive Bayes são um grupo de algoritmos de classificação **extremamente rápidos e simples** que costumam ser **adequados para conjuntos de dados de dimensões muito altas**.

Naive Bayes (NB)

Os modelos baseados em Naive Bayes são um grupo de algoritmos de classificação **extremamente rápidos e simples** que costumam ser **adequados para conjuntos de dados de dimensões muito altas**.

Por serem rápidos e terem **poucos (ou nenhum) parâmetros ajustáveis**, estes acabam sendo muito úteis como método de comparação (**baseline**) para qualquer problema de classificação.

Naive Bayes (NB)

Os modelos baseados em Naive Bayes são um grupo de algoritmos de classificação **extremamente rápidos e simples** que costumam ser **adequados para conjuntos de dados de dimensões muito altas**.

Por serem rápidos e terem **poucos (ou nenhum) parâmetros ajustáveis**, estes acabam sendo muito úteis como método de comparação (**baseline**) para qualquer problema de classificação.

Esta aula tentará ser uma explicação intuitiva sobre como as **diferentes versões** dos classificadores Naive Bayes, seguida por alguns exemplos deles.

Classificação Bayesiana

O classificador de Naive Bayes é **baseado no teorema de Bayes**, que é uma equação que descreve uma **relação entre probabilidades condicionais e a priori**.

Na classificação bayesiana, estamos interessados em calcular a probabilidade de um rótulo dadas algumas características (features) observadas **$P(L|\text{features})$** (likelihood).

O teorema de Bayes diz como expressar isso em quantidades que podemos calcular diretamente como:

$$P(L|\text{features}) = \frac{P(\text{features}|L)P(L)}{P(\text{features})}$$

Aplicação da regra

Suponha que a probabilidade observar um paciente com uma doença Y é de 0,08 (8%) e que existe um teste cujo resultado não é perfeito, mas segue a tabela:

Teste	Doença	
	presente	ausente
positivo	75%	4%
negativo	25%	96%

$P(\text{teste=negativo} \mid \text{doença} = \text{ausente})$

Probabilidades a priori:

$$P(\text{doença=presente}) = 0,08$$

$$P(\text{doença=ausente}) = 1 - 0,08 = 0,92$$

Lei da probabilidade total
$$P(X) = \sum_{i=1}^n P(X|y_i)P(y_i)$$

Aplicação da regra

Suponha que a probabilidade observar um paciente com uma doença Y é de 0,08 (8%) e que existe um teste cujo resultado não é perfeito, mas segue a tabela:

Teste	Doença	
	presente	ausente
positivo	75%	4%
negativo	25%	96%

$P(\text{teste=negativo} \mid \text{doença} = \text{ausente})$

Probabilidades a priori:

$$P(\text{doença=presente}) = 0,08$$

$$P(\text{doença=ausente}) = 1 - 0,08 = 0,92$$

Lei da probabilidade total
$$P(X) = \sum_{i=1}^n P(X|y_i)P(y_i)$$

$$P(\text{teste=pos}) = P(\text{pos|pres}) P(\text{pres}) + P(\text{pos|aus}) P(\text{aus}) = 0,75 \cdot 0,08 + 0,04 \cdot 0,92 = 0,0968$$

$$P(\text{teste=neg}) = P(\text{neg|pres}) P(\text{pres}) + P(\text{neg|aus}) P(\text{aus}) = 0,25 \cdot 0,08 + 0,96 \cdot 0,92 = 0,9032$$

Aplicação da regra

Suponha que a probabilidade observar um paciente com uma doença Y é de 0,08 (8%) e que existe um teste cujo resultado não é perfeito, mas segue a tabela:

Teste	Doença	
	presente	ausente
positivo	75%	4%
negativo	25%	96%

$P(\text{teste=negativo} \mid \text{doença} = \text{ausente})$

Probabilidades a priori:

$$P(\text{doença=presente}) = 0,08$$

$$P(\text{doença=ausente}) = 1 - 0,08 = 0,92$$

Lei da probabilidade total
$$P(X) = \sum_{i=1}^n P(X|y_i)P(y_i)$$

$$P(\text{teste=pos}) = P(\text{pos|pres}) P(\text{pres}) + P(\text{pos|aus}) P(\text{aus}) = 0,75 \cdot 0,08 + 0,04 \cdot 0,92 = 0,0968$$

$$P(\text{teste=neg}) = P(\text{neg|pres}) P(\text{pres}) + P(\text{neg|aus}) P(\text{aus}) = 0,25 \cdot 0,08 + 0,96 \cdot 0,92 = 0,9032$$

Mas, de onde
saíram os valores
da tabela?



Aplicação da regra

Suponha que a probabilidade observar um paciente com uma doença Y é de 0,08 (8%) e que existe um teste cujo resultado não é perfeito, mas segue a tabela:

Teste	Doença	
	presente	ausente
positivo	75%	4%
negativo	25%	96%

P(teste=negativo | doença = ausente)

Probabilidades a priori:

$$P(\text{doença=presente}) = 0,08$$

$$P(\text{doença=ausente}) = 1 - 0,08 = 0,92$$

x (teste)	Label
Neg.	Pres.
Neg.	Aus.
Neg.	Aus.
...	
Pos.	Pres.
Pos.	Aus.

Lei da probabilidade total
$$P(X) = \sum_{i=1}^n P(X|y_i)P(y_i)$$

$$P(\text{teste=pos}) = P(\text{pos|pres}) P(\text{pres}) + P(\text{pos|aus}) P(\text{aus}) = 0,75 \cdot 0,08 + 0,04 \cdot 0,92 = 0,0968$$

$$P(\text{teste=neg}) = P(\text{neg|pres}) P(\text{pres}) + P(\text{neg|aus}) P(\text{aus}) = 0,25 \cdot 0,08 + 0,96 \cdot 0,92 = 0,9032$$



Aplicação da regra

Uma vez que temos todos os ingredientes podemos voltar e aplicar a regra de decisão **MAP** (Maximum a posteriori):

$$y_{MAP} = \arg \max_i P(y_i|x)$$

Que se traduz em encontrar o y_i que maximiza:

$$P(y_i|x) = \frac{P(y_i)P(x|y_i)}{P(x)}$$

Observem que **o denominador $P(x)$** não depende da classe y_i , e portanto torna-se um termo de normalização que **pode ser ignorado**:

$$P(y_i|x) \propto P(y_i)P(x|y_i)$$

Aplicação da regra

Mas, e como tratar **vetores de atributos x** ?

Aplicação da regra

Mas, e como tratar **vetores de atributos** $\mathbf{x}$?

Assumindo que os valores dos **atributos são independentes**, então podemos reescrever:

$$P(y_i|\mathbf{x}) \propto P(y_i) \prod_{j=1}^d P(\mathbf{x}_j|y_i)$$

onde d é o total de atributos (features).

Aplicação da regra

Mas, e como tratar **vetores de atributos $\mathbf{x}$** ?

Assumindo que os valores dos **atributos são independentes**, então podemos reescrever:

$$P(y_i|\mathbf{x}) \propto P(y_i) \prod_{j=1}^d P(\mathbf{x}_j|y_i)$$

onde d é o total de atributos (features).

Computacionalmente o produto pode causar erros numéricos, então seria melhor aplicar uma **transformação logarítmica**:

$$\log(P(y_i|\mathbf{x})) \propto \log(P(y_i)) \sum_{j=1}^d \log(P(\mathbf{x}_j|y_i))$$

Considerações

1. Em problemas binários com atributos booleanos o **NB gera um hiperplano de separação linear**.
2. Na prática **ignorar essa restrição dos atributos independentes não causa muitos problemas** na classificação, porém **é fundamental evitar features redundantes**.
3. **Todas as probabilidades necessárias podem ser calculadas a partir do conjunto de treinamento**, portanto este deve ser representativo das probabilidades de ocorrências das classes $P(y_i)$.
4. Computacionalmente simples, rápido, eficiente e **robusto à presença de ruídos e atributos irrelevantes**.

Considerações

5. Para calcular as $P(\mathbf{x}_i|y_i)$ é necessário distinguir entre **atributos discretos (nominais) e contínuos**:
 - a. Para atributos nominais basta manter um contador de ocorrência para cada classe (abordagem frequentista);
 - b. No caso de **atributos com valores reais**, onde existem infinitos valores possíveis, existem duas possibilidades:
 - i. Assumir que os valores vem de uma **distribuição normal** $P(\mathbf{x}_i|y_i) \sim N(mu, sigma)$, na qual devem ser determinados os parâmetros mu e $sigma$
 - ii. Ou **discretizar os valores** da variável real em bins (intervalos discretos) para poder aplicar a regra frequentista

Prática no Sklearn

Diferentes alternativas do Sklearn:

- **Categorical NB**
- Gaussian NB
- Bernoulli NB
- Multinomial NB
- etc.

São apropriados para diferentes tipos de features e distribuições, por exemplo Gaussian NB é útil para atributos com números reais, enquanto que Categorical NB é recomendado para variáveis categóricas.

https://scikit-learn.org/stable/modules/naive_bayes.html

Prática no Sklearn

Neste exemplo vamos utilizar:

- Categorical NB
- O dataset Car Evaluation
- Fazer download (<https://archive.ics.uci.edu/ml/datasets/Car+Evaluation>)
- Jupyter Notebook (colab.research, kaggle, etc.)
 - No caso de usar o colab, o dataset pode estar numa pasta do google drive

Exemplo: https://colab.research.google.com/drive/1RFxf6w5GFeldBTADTetkubuBH_fjXMcX?usp=sharing

Atividade proposta

Neste exemplo vamos utilizar:

- Escolher um classificador do tipo NB (apropriado)
- Utilizar o dataset Congressional Voting Records
(<https://archive.ics.uci.edu/ml/datasets/Congressional+Voting+Records>)
 - Descrever os dados com as estatísticas principais
 - Conversão de features
 - Tratamento das missing features (simples remoção de linhas da tabela)
 - Train-Test split
- Criar um Jupyter Notebook (colab.research, kaggle, etc.)
- Mostrar o desempenho com várias métricas (confusion matrix, etc.)
- Enviar a atividade pelo Classroom

Segunda parte

Gaussian Naive Bayes

Como obter as probabilidades do Naive Bayes quando os atributos são números reais?

Obviamente não podemos usar uma abordagem por contagem (frequentista).

Gaussian Naive Bayes

Se você tentar passar dados numéricos contínuos diretamente para um classificador CategoricalNB (como a implementação do Scikit-learn), **ele tratará cada valor real único como uma categoria distinta.**

Problemas:

1. Perda de Generalização: Valores como 3.1415, 3.1416 e 3.1417 são muito próximos e provavelmente contêm informações semelhantes. O CategoricalNB os tratará como três categorias completamente diferentes e não relacionadas, assim como trataria "vermelho", "azul" e "verde".
2. Problema de Frequência Zero (Zero-Frequency Problem): Durante a previsão, se o modelo encontrar um valor numérico que não estava presente exatamente da mesma forma no conjunto de treinamento, ele não terá nenhuma probabilidade calculada para essa "categoria" e não conseguirá fazer uma previsão adequada.

Gaussian Naive Bayes

Existem duas formas de resolver esse dilema:

1. Usar a Variante Correta do Naive Bayes (GaussianNB):
 - a. Para cada feature e cada classe no conjunto de dados de treino, ele calcula a média (μ) e o desvio padrão (σ).
 - b. Usa a Função de Densidade de Probabilidade (PDF) Gaussiana para calcular a probabilidade de um determinado valor numérico pertencer a cada classe, com base na média e no desvio padrão daquela classe.

Onde:

$$P(x_i|y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} \exp\left(-\frac{(x_i - \mu_y)^2}{2\sigma_y^2}\right)$$

- x_i é o valor da feature.
- y é a classe.
- μ_y é a média da feature x_i para a classe y .
- σ_y^2 é a variância (desvio padrão ao quadrado) da feature x_i para a classe y .

Binning

Existem duas formas de resolver esse dilema:

1. Discretização (ou Binning) das Features: podemos transformar as features numéricas em categóricas. A ideia é agrupar os números em 'intervalos'.

Exemplo: Imagine que você tem uma feature Idade com valores de 1 a 100. Em vez de usar os números diretamente, você pode criar categorias:

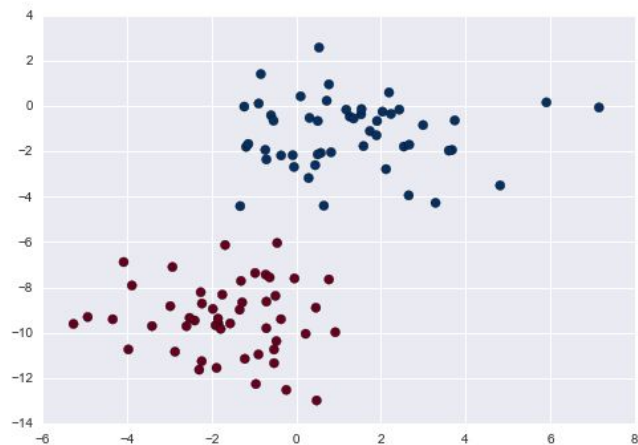
- A. 0-17: "Criança/Adolescente"
- B. 18-35: "Jovem Adulto"
- C. 36-60: "Meia-idade"
- D. 61+: "Idoso"

No Scikit-learn, você pode fazer isso facilmente usando **KBinsDiscretizer** ou a função `cut` do Pandas antes de treinar o modelo.

Gaussian Naive Bayes

Naive Bayes gaussiano assume que os dados de cada rótulo são extraídos de uma distribuição gaussiana simples.

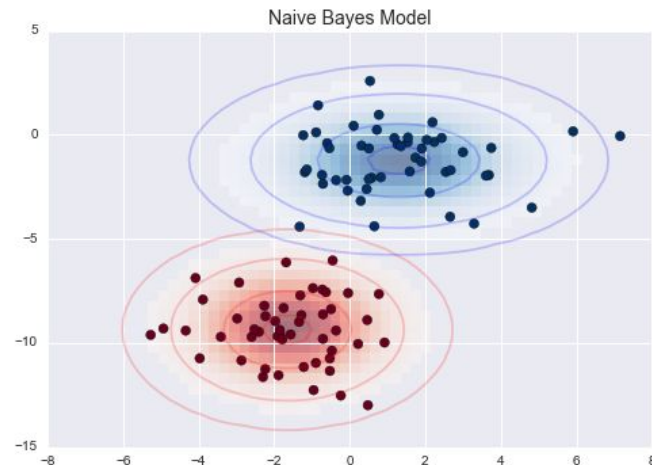
Imagine que você tem os seguintes dados:



O Sklearn possui alguns métodos para gerar datasets sintéticos com distribuições específicas (<https://scikit-learn.org/stable/datasets.html>)

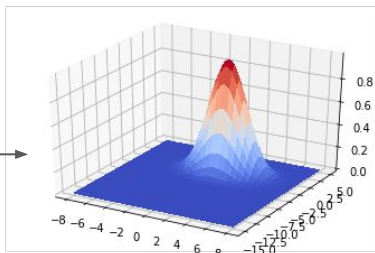
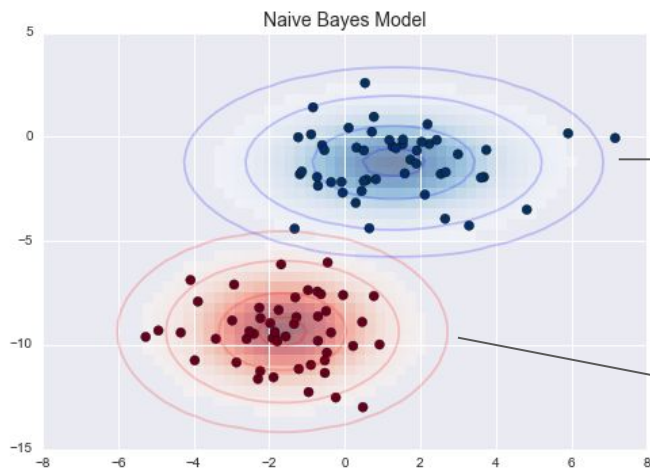
Gaussian Naive Bayes

Uma maneira de criar um modelo simples é assumir que o modelo pode ser ajustado simplesmente encontrando a média e o desvio padrão dos pontos dentro de cada rótulo.

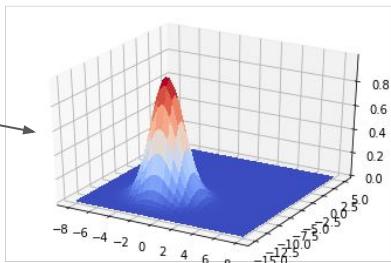


Gaussian Naive Bayes

As elipses aqui representam o modelo generativo gaussiano para cada rótulo, com maior probabilidade em direção ao centro das elipses. Com este modelo temos uma receita simples para calcular a probabilidade $P(\text{atributo}|L)$ para qualquer ponto.



$$P(x_i|y_1) = \exp\left(-\frac{(x_i - \mu_{y_1})^2}{2\sigma_{y_1}^2}\right)$$



$$P(x_i|y_2) = \exp\left(-\frac{(x_i - \mu_{y_2})^2}{2\sigma_{y_2}^2}\right)$$

Gaussian Naive Bayes

Exemplo: <https://colab.research.google.com/drive/1LjJRMVum-QebPGOgigEubrYGtaLmsA8B?usp=sharing>

Considerações:

- Vemos uma função de decisão ligeiramente curvada que separa as classes, isto mostra que em geral **NB gaussiano é quadrático**.
- Uma boa parte desse formalismo bayesiano é que ele permite, naturalmente, **gerar uma classificação probabilística**, que podemos calcular usando o método **predict_proba()**.
- A classificação final será tão boa quanto as suposições do modelo que levam a ela, e é por isso que este modelo muitas vezes não produz resultados muito bons.
 - Entretanto, especialmente quando o **número de atributos é grande, descumprir a premissa Gaussiana não é prejudicial o suficiente** para impedir que o método seja útil.

Terceira parte

Multinomial Naive Bayes

Até agora cobrimos os casos nos quais os atributos são categóricos ou reais, mas como tratar casos em que os valores dos atributos correspondem a uma contagem de um evento?

Por exemplo, considere as duas frases:

1. Um cachorro late
2. Um cachorro late e um outro cachorro morde.

Um vetor de atributos possíveis para estas duas frases pode considerar a frequência de ocorrência de cada palavra, por exemplo

	um	dois			gato	cachorro	carro	o	
1.	1	0			0	1	0	0	
2.	2	0			0	2	0	0	

Multinomial Naive Bayes

Até agora cobrimos os casos nos quais os atributos são categóricos ou reais, mas como tratar casos em que os valores dos atributos correspondem a uma contagem de um evento?

Por exemplo, considere as duas frases:

1. Um cachorro late
2. Um cachorro late e um outro cachorro morde.

Um vetor de atributos possíveis para estas duas frases pode considerar cada palavra, por exemplo

Abordagem também conhecida como bag-of-words.

	um	dois			gato	cachorro	carro	o	
1.	1	0			0	1	0	0	
2.	2	0			0	2	0	0	

Multinomial Naive Bayes

Uma distribuição multinomial é uma generalização de uma distribuição binomial na qual existem vários eventos (mutuamente exclusivos x_i) ocorrendo em um mesmo espaço amostral.

$$P(x_1, x_2, \dots, x_k) = \frac{n!}{x_1! x_2! \dots x_k!} (p_1)^{x_1} (p_2)^{x_2} \dots (p_k)^{x_k}$$

onde n é a quantidade de realizações, k é a quantidade de alternativas (eventos possíveis diferentes), p_i a probabilidade de cada evento e $p_1 + p_2 + \dots + p_k = 1$.

“A distribuição multinomial descreve a probabilidade de observar contagens entre várias categorias e, portanto, é mais apropriada para atributos que representam contagens”

Multinomial Naive Bayes

Com o Naive Bayes Multinomial a ideia é similar, exceto que, em vez de modelar a distribuição de dados com uma Gaussiana, modelamos a distribuição dos dados com uma distribuição multinomial.

- O sklearn possui a versão MultinomialNB().
- Similarmente à versão CategoricalNB() existe um parâmetro alpha de suavização que evita contornar o problema de algumas probabilidades serem zero por causa de algum evento não estar representado no conjunto de dados.

Exemplo: <https://colab.research.google.com/drive/1qDCThxfXD-SrPdP5woxrbANDwNpvphCI?usp=sharing>

Multinomial Naive Bayes

Para lidar com dados textuais temos várias alternativas:

- vetores que contam a frequência dos termos (como na tabela anterior)
- vetores binário que representam a ocorrência do termo (sem considerar a contagem)
- TF-IDF
- Word2Vec
- GloVe
- etc....

TF-IDF

Wikipedia: O valor TF-IDF (term frequency-inverse document frequency), é uma medida estatística que indica a importância de uma palavra de um documento em relação a uma coleção de documentos. Ela é frequentemente utilizada na recuperação de informações e mineração de dados.

O TF-IDF de uma palavra aumenta proporcionalmente à medida que aumenta o número de ocorrências dela em um documento (d), no entanto, esse valor é equilibrado pela frequência da palavra no corpus. Isso auxilia a distinguir o fato da ocorrência de algumas palavras serem geralmente mais comuns que outras.

$$\text{tf}(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$$

$$\text{idf}(t, D) = \log \frac{N}{|\{d \in D : t \in d\}|}$$

$$\text{tfidf}(t, d, D) = \text{tf}(t, d) \cdot \text{idf}(t, D)$$

TF-IDF

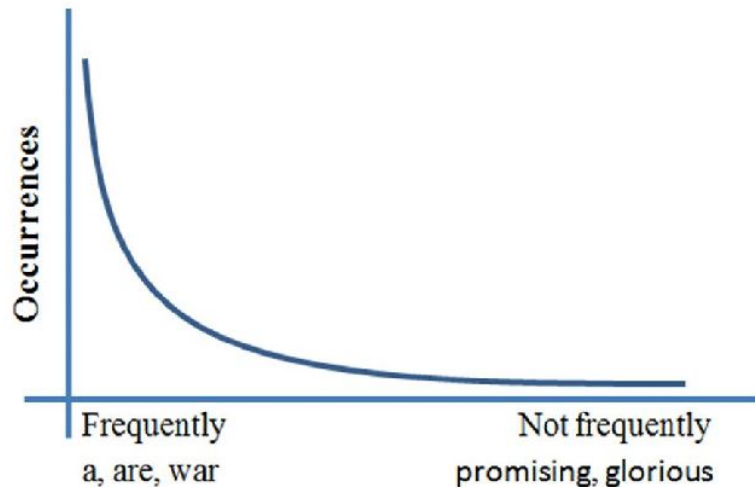
Exemplo: deseja-se recuperar documentos relevantes com a frase "the brown cow".

Todos os documentos com estes três termos são soluções relevantes, entretanto em alguns documentos estes três termos podem aparecer mais de uma vez, inclusive a palavra "the" vai aparecer muitas vezes causando a impressão errada de ser um termo mais relevante do que "brown cow"...

O IDF mede a quantidade de informação que o termo proporciona, então a multiplicação

$$\text{tfidf}(t, d, D) = \text{tf}(t, d) \cdot \text{idf}(t, D)$$

pondera a informação vezes a frequência, equilibrando o score TF-IDF para os termos frequentes.



Exemplo Prático e Simples

Imagine um corpus com 2 documentos:

- **Documento 1:** "O cão persegue o gato."
- **Documento 2:** "O gato foge do leão."

Vamos calcular o TF-IDF da palavra **"gato"** no **Documento 1**.

1. **Calcular o TF de "gato" no Documento 1:**
 - "gato" aparece 1 vez. O documento tem 5 palavras.
 - $TF = 1 / 5 = 0.2$
2. **Calcular o IDF de "gato":**
 - Temos 2 documentos no total.
 - A palavra "gato" aparece em 2 documentos.
 - $IDF = \log(2 / 2) = \log(1) = 0$
3. **Calcular o TF-IDF final:**
 - $TF-IDF("gato", Doc1) = 0.2 * 0 = 0$
 - *Resultado: "gato" não é uma palavra distintiva neste corpus, pois aparece em todos os documentos.*

calcular o TF-IDF da palavra **"cão"** no **Documento 1**.

1. **Calcular o TF de "cão" no Documento 1:**
 - "cão" aparece 1 vez. O documento tem 5 palavras.
 - $TF = 1 / 5 = 0.2$
2. **Calcular o IDF de "cão":**
 - Temos 2 documentos no total.
 - A palavra "cão" aparece em apenas 1 documento (o Documento 1).
 - $IDF = \log(2 / 1) = \log(2) \approx 0.301$
3. **Calcular o TF-IDF final:**
 - $TF-IDF("cão", Doc1) = 0.2 * 0.301 = 0.0602$
 - *Resultado: "cão" tem uma pontuação maior que "gato" porque é uma palavra mais distintiva do Documento 1 dentro deste pequeno corpus.*

Conclusão: se tivermos um classificador que precisa separar os dois documentos, simplesmente olhando a ocorrência da palavra 'cão' seria suficiente.

Atividade proposta

Classificar o conjunto 20 newsgroup:

- Usar codificação binária para as palavras (ao invés de contagem).
- Utilizar as mesmas classes do exemplo para que os resultados sejam comparáveis.
- Escolher a versão mais apropriada do Naive Bayes para lidar com atributos binários.
- Testar diferentes configurações, por exemplo: com ou sem stop words, com ou sem o cabeçalho das notícias, etc.
- Um aluno será sorteado para mostrar e explicar o que foi feito.
- Enviar a atividade

Considerações finais

Desvantagens:

- Pelo fato de termos suposições fortes sobre os dados, o NB geralmente não tem um desempenho tão bom comparado a outros modelos.

Vantagens:

- Eles são extremamente rápidos para treinamento e previsão
- Eles fornecem uma previsão probabilística direta (embora algumas vezes não seja muito confiável)
- Muitas vezes são facilmente interpretáveis
- Eles têm muito poucos (se houver) parâmetros ajustáveis

Essas vantagens significam que um classificador Bayesiano ingênuo costuma ser uma boa escolha como classificador inicial.

Considerações finais

Geralmente são bons quando:

- Quando as suposições realmente correspondem aos dados (muito raro na prática)
- Para categorias muito bem separadas, quando a complexidade do modelo é menos importante
- Para dados de dimensões muito altas, quando a complexidade do modelo é menos importante

Os dois últimos pontos parecem distintos, mas na verdade estão relacionados: conforme a dimensão de um conjunto de dados cresce, é muito menos provável que quaisquer dois pontos sejam encontrados próximos (afinal, eles devem estar próximos em todas as dimensões para estarem próximos no geral).

Isso significa que em dimensões altas os dados tendem a ser mais separados. Por esse motivo, classificadores simplistas como NB tendem a funcionar tão bem ou melhor do que classificadores mais complicados conforme a dimensionalidade aumenta.

Aprendizado: “Tendo dados suficientes, até mesmo um modelo simples pode ser muito poderoso”

Tarefa

Usar o conjunto 20 news groups e validação cruzada 10-CV para comparar o classificador MultinomialNB com e sem priors.